

Livello del sistema operativo

GESTIONE DEI PROCESSI

Introduzione

Multiprogrammazione:
concorrenza dell'esecuzione dei programmi

Numero dei programmi in esecuzione maggiore del numero di CPU

- Virtualizzazione della CPU: ogni processo dispone di un PC e di un set di registri (contesto)

Kernel: componente del S.O. che ha il compito di realizzare la astrazione di CPU virtuale

Traccia dei programmi in esecuzione:
introduzione del concetto di processo

Definizione di processo

Programma: entità statica

Esecuzione di un programma → generazione di una sequenza di eventi

- Ogni evento corrisponde all'esecuzione di una operazione e determina un cambiamento di stato nell'elaboratore

Processo = sequenza di eventi generati dall'esecuzione di un programma

- Entità dinamica
- Istanza di programma
- Caratterizzabile dalla sequenza degli stati in cui passa un elaboratore durante l'esecuzione di un programma

3

Programma vs. Processo

```
{  
  a=x; b=y;  
  while (a!=b)  
    if (a>b) a=a-b;  
    else b=b-a;  
}
```

Programma

Processo

	t ₀	t ₁	t ₂	t ₃	t ₄	t ₅
x	18	18	18	18	18	18
y	24	24	24	24	24	24
a	?	18	18	18	12	6
b	?	?	24	6	6	6

4

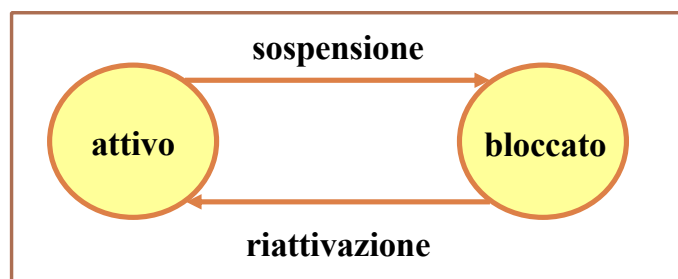
Nucleo di un sistema operativo

- Il kernel è responsabile (fra le altre cose) della gestione dei processi
- Realizza l'astrazione di CPU virtuale:
 - Creazione e terminazione di processi
 - Sincronizzazione di processi
 - Scheduling della CPU
 - Cambio di contesto
 - Risposta a eventi esterni (interrupt)
- Utilizza strutture dati aggiuntive
 - Descrittori di processi
 - Code di processi
- Richiede una realizzazione estremamente efficiente

5

Stati di un processo (1)

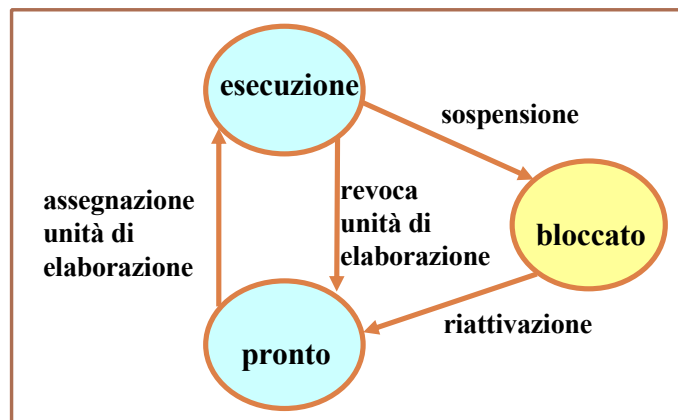
- Un processo in esecuzione è soggetto a **transizioni di stato**
 - Definite da eventi esterni e dall'attività del processo stesso
- Ipotesi iniziale: numero processi = numero processori
- Due stati possibili
 - Attivo: CPU esegue le istruzioni del programma
 - Bloccato: Stato di attesa di un evento esterno
 - Attesa di un dispositivo, di un messaggio, sincronizzazione, etc.



6

Stati di un processo (2)

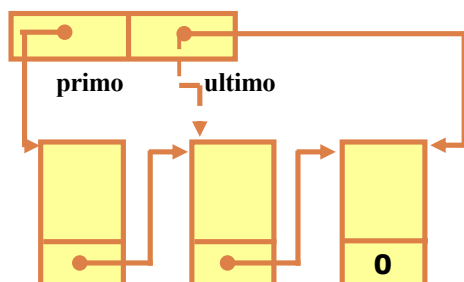
- Ipotesi più realistica: numero processi > numero processori
- Stato pronto: processo pronto per essere eseguito ma nessun processore è disponibile
 - Coda di processi



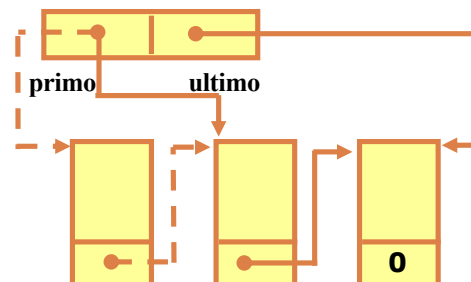
7

Code di processi

- Diverse strutture dati per fare riferimento ai processi in esecuzione
 - Coda dei processi pronti
 - Coda dei processi in attesa
- Processo idle (processo fittizio, dummy process)



Operazione di inserimento

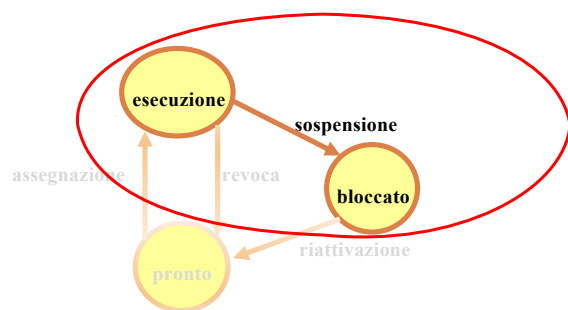
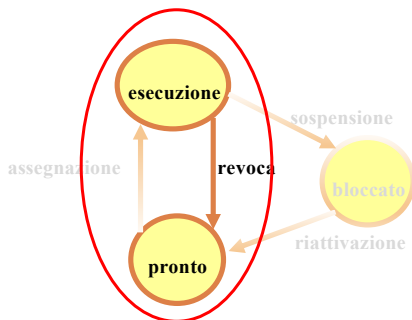


Operazione di prelievo

8

Scheduling – Quando

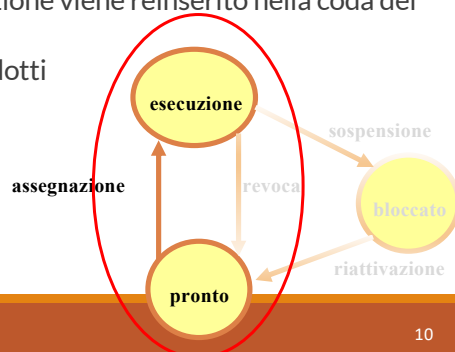
- Preemptive (**con pre-rilascio**)
 - Lo scheduler interviene per decidere di revocare la CPU a un processo in esecuzione
 - Scadenza del quanto di tempo
 - Esecuzione di processi a priorità più elevata
- Non preemptive (**senza pre-rilascio**)
 - Lo scheduler interviene esclusivamente quando il processo libera spontaneamente la CPU
 - Accesso a risorse (sospensione in attesa)
 - Rilascio volontario (terminazione)



9

Scheduling – Come

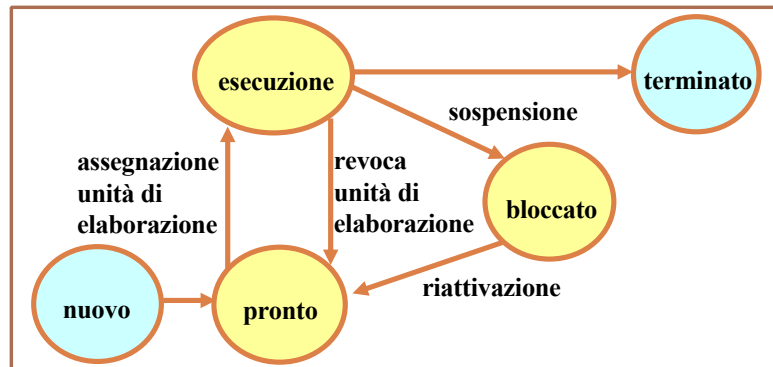
- **First Come First Served**
 - CPU assegnata al processo da più tempo in coda
 - Tipico dello scheduling senza pre-rilascio
- **A priorità**
 - CPU assegnata al processo con priorità più alta
 - Problema: **starvation** (soluzione: **aging**)
- **Round robin**
 - A ogni processo in esecuzione è assegnato un quanto di tempo
 - Allo scadere del quanto, il processo in esecuzione viene reinserto nella coda dei processi pronti
 - Overhead aumentato per quanti di tempo ridotti



10

Stati di un processo (3)

- Ulteriori stati generali
 - Stato nuovo: creazione di un nuovo processo
 - Stato terminato: terminazione di un processo
 - T. normale
 - T. anomala
- (Mancano) ulteriori stati specifici del S.O.



11

Creazione e terminazione dei processi

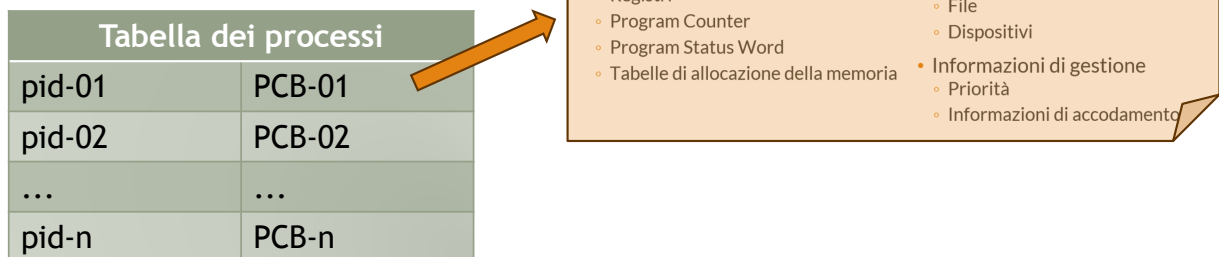
- Creazione di processi
 - Un processo richiede la creazione di un processo figlio al SO
 - Es. chiamata di sistema Unix: **fork()**
 - Gerarchia di processi (creazione di un albero dei processi)
 - Inizializzazione del descrittore e allocazione delle risorse
- Terminazione di processi
 - Un processo chiede di terminare
 - Un processo viene terminato (ucciso)
 - Es. chiamata di sistema Unix: **kill()**
 - De-allocazione delle risorse

12

Descrizione dei processi

- A ogni processo è associata una struttura dati:
 - **Descrittore di Processo** (Process Control Block, PCB)
- Tutti i descrittori dei processi sono raccolti in una tabella:
 - **Tabella dei processi** (Process Table)

Tabella dei processi	
pid-01	PCB-01
pid-02	PCB-02
...	...
pid-n	PCB-n

- 
- Nome del processo (PID)
 - Identificatore univoco
 - Stato del processo
 - Contesto del processo
 - Registri
 - Program Counter
 - Program Status Word
 - Tabelle di allocazione della memoria
 - Risorse assegnate
 - Memoria
 - Codice
 - Dati
 - Stack
 - File
 - Dispositivi
 - Informazioni di gestione
 - Priorità
 - Informazioni di accodamento

13

Concorrenza tra processi (1)

La concorrenza è una relazione tra processi

Due processi P e Q si dicono concorrenti quando la prima operazione di P inizia prima dell'ultima operazione di Q o viceversa

- Processi concorrenti → Processi la cui esecuzione si sovrappone nel tempo

Due forme di concorrenza

- Overlapping: esecuzione parallela
 - Ogni processo possiede una propria unità di elaborazione
- Interleaving: esecuzione in condivisione di tempo
 - I processi condividono la stessa unità di elaborazione

14

Concorrenza tra processi (2)

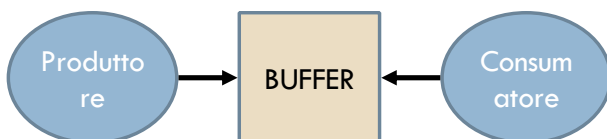
- Proprietà dei processi concorrenti
 - Processi indipendenti: non condividono dati e non si scambiano informazioni
 - Nessuno di essi può influenzare l'esecuzione degli altri
 - Comportamento riproducibile: la sequenza degli eventi dipende unicamente dal programma in esecuzione e dai dati forniti
 - Lo stesso processo, eseguito in momenti diversi insieme a diversi processi indipendenti, produce sempre lo stesso risultato
 - Processi interagenti: esecuzione influenzata vicendevolmente
 - Comportamento non riproducibile

15

Interazione tra processi

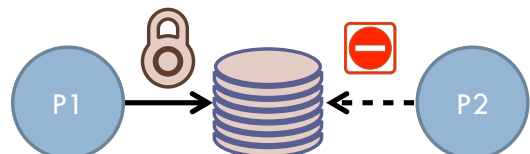
COOPERAZIONE

- Accesso a una risorsa condivisibile
 - Es. buffer
- Necessario stabilire un **ordine di accesso**
 - Es: num. inserimenti nel buffer $\geq$ num. di prelievi
- Sincronizzazione **diretta** (o **esplicita**)
 - Processi necessariamente interagenti



COMPETIZIONE

- Accesso contemporaneo a una risorsa **NON** condivisibile
 - Es. accesso al disco
- Necessaria **virtualizzazione**
 - Es: i processi accedono al disco come se fosse assegnato solo a essi
- Sincronizzazione **indiretta** (o **implicita**)
 - Garanzia che un solo processo per volta acceda alla risorsa



16

Cambio di contesto

- Quando la CPU viene rilasciata da un processo e assegnata a un altro occorre effettuare una serie di operazioni:
 1. Salvataggio del contesto nel Process Control Block
 2. Accodamento del processo
 3. Selezione del nuovo processo
 4. Caricamento del nome del nuovo processo nel registro dei processi in esecuzione
 5. Caricamento del nuovo contesto
- Operazione **onerosa**

17

Cambio di contesto

- Indica l'insieme delle operazioni che vengono svolte per ri-assegnare la CPU a un altro processo
 - Molto frequente nei sistemi interattivi (centinaia di cambi al secondo)
- Ogni cambio di contesto implica:
 - Salvataggio in memoria centrale dello stato della CPU del processo revocato (contesto)
 - Registri generali + PSW + PC + Tabelle di allocazione della memoria + puntatore al descrittore + ...
 - Caricamento nella CPU del contesto del processo assegnato
 - Registri generali + PSW + PC + Tabelle di allocazione della memoria + puntatore al descrittore + ...
- Cambio di contesto → overhead elevato!

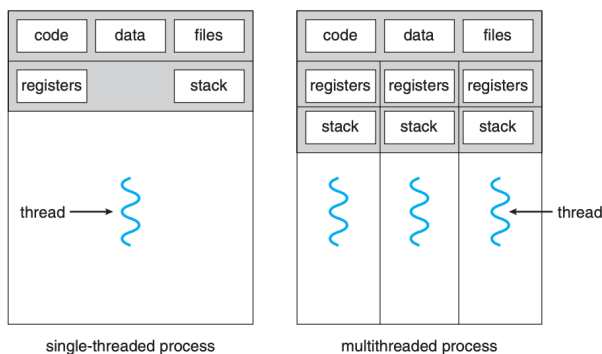
18

Applicazioni multi-thread

- Sono numerose le applicazioni con un elevato grado di parallelismo al loro interno:
 - Es. Browser web, word processor, applicazioni grafiche, sistemi di controllo, ...
- Per queste applicazioni, sussistono molte attività (**thread**) concorrenti che condividono molte risorse
 - Spazio di indirizzamento, risorse hw, file, etc.
- L'overhead è minore se il cambio di contesto avviene nell'ambito della stessa applicazione
 - Cambio di contesto sostituisce solo le porzioni di stato che non sono condivise (es. registri generali, PC, PSW)

19

Applicazioni multi-thread



ESEMPI

- Web browser: un thread per mostrare immagini o testo, un thread per recuperare dati dalla rete
 - Word processor: un thread per mostrare grafici, un thread per gestire l'immissione del testo, un thread per il correttore ortografico
- **Thread** = flusso di esecuzione all'interno di un processo
 - **Processo** = risorse + {thread(s)}
 - Singolo thread
 - Multithread

20

Vantaggi nell'utilizzo dei thread

- **Condivisione di risorse**
 - I thread condividono memoria e risorse di uno stesso processo
 - Risulta semplificata la comunicazione fra thread
- **Efficienza**
 - Cambio di contesto semplificato
 - Costi di creazione e di de-allocazione delle risorse ridotti
- **Scalabilità**
 - Nelle architetture multi-processori o multi-core i threads possono operare in parallelo
 - Un processo a singolo thread può girare su un solo processore
- **Reattività**
 - Un'applicazione interattiva continua a girare anche se una sua parte è bloccata o impegnata in elaborazioni onerose
 - Aumento di reattività a beneficio dell'utente